

Testing Policy Document

Neighbourhood WatchDog

Version: 1.1

Last Updated: 31 July 2026

1. What This Document Is For

This document exists so everyone on the team knows how we test the Neighbourhood WatchDog system, what kinds of tests we write, and where they live in the codebase.

The goal is simple: every time someone pushes code, we want to feel reasonably confident that the system still works. Tests are how we get that confidence.

2. What We Are Actually Testing

Our system has three main parts, and we test all of them:

Part	What It Does	Test Folder
Backend API	Handles alerts, cameras, neighbourhoods, user auth, and detection events	tests/backend/
Frontend UI	React pages that users interact with (alerts, join requests, etc.)	tests/frontend/
End-to-End Flows	Full user journeys from opening the browser to completing a task	tests/e2e/

3. Test Types and Where They Go

3.1 Unit Tests

Unit tests check one small piece of code in isolation. If a test fails, you should know exactly which function broke.

We split our backend unit tests into two folders:

3.1.1 Service Tests (`tests/backend/unit/services/`)

These test the business logic inside our service layer. A service function is the thing that actually does the work, like creating a neighbourhood or acknowledging an alert. We mock the database and any external calls so the test only cares about the logic.

What we test:

- Happy path (everything goes right)
- Missing or invalid inputs (empty strings, None values, wrong types)
- Missing database connection
- Missing or wrong authentication claims
- Permission checks (e.g. a resident trying to do an admin action)
- Duplicate or conflicting states (e.g. trying to acknowledge an already acknowledged alert)

How we write them:

- Each test class uses `setup_method()` to create a mock database and fake user claims
- Each test uses `teardown_method()` to clean up any patches
- We use `unittest.mock.Mock` for the database and `pytest.raises(HTTPException)` to check error cases
- Every test name should describe what it checks, e.g. `test_missing_alert_id_raises_400`

3.1.2 Model / Schema Tests (`tests/backend/unit/models/`)

These test our Pydantic schemas and response models. Schemas define what data looks like when it moves in and out of the API. If a schema rejects bad data, the API never even sees it.

What we test:

- Valid data passes validation (happy path)
- Missing required fields raise `ValidationError`
- Invalid types raise `ValidationError` (e.g. a string where a UUID is expected)
- Boundary values (e.g. confidence score of 0.0 and 1.0 are fine, but 1.01 is not)
- Optional fields default to `None` when not provided
- Nested objects validate correctly inside response wrappers

How we write them:

- Use helper functions like `_make_alert_res()` to build valid base data, then override specific fields

- Use `pytest.mark.parametrize` when testing multiple valid values (e.g. all allowed detection types)
- Every test should have a docstring explaining what scenario it covers

3.2 Integration Tests (`tests/backend/integration/api/`)

Integration tests check that our API endpoints work from the outside. They make fake HTTP requests to the FastAPI app and check the response status codes and bodies.

What we test:

- Endpoints return the right status codes (200, 201, 401, etc.)
- Request payloads are accepted and responses contain expected data
- Authentication headers are required where needed
- Multiple related endpoints work together (e.g. list alerts then acknowledge one)

How we write them:

- Use `async_client` and `auth_headers` fixtures (provided by `conftest.py`, not shown here)
- Mock the service layer with `AsyncMock` so the test does not need a real database
- Use constants for repeated values like timestamps
- Group related endpoint tests in the same file

3.3 Frontend Unit Tests (`tests/frontend/unit/`)

These test React components in isolation using React Testing Library and Jest. We render a page, simulate user actions, and check that the UI updates correctly.

What we test:

- Pages load and display expected text
- Forms accept input and submit correctly
- API calls are made with the right arguments
- Error states show error messages
- WebSocket messages update the UI in real time

How we write them:

- Mock API modules with `jest.mock()`
- Mock `next/navigation` hooks
- Mock `WebSocket` globally for real-time features
- Use `waitFor()` for async UI updates
- Use fixtures (see below) for consistent test data

3.4 Frontend Fixtures (tests/frontend/fixtures/)

Fixtures are reusable bits of fake data. Instead of writing the same mock alert object in ten different tests, we define it once and import it everywhere.

Example:

```
import type { Alert } from "@components/shared/AlertCard";
export const mockAlert: Alert = {
  id: "alert-1",
  camera_id: "cam-1234",
  detection_event_id: "evt-1",
  status: "NEW",
  created_at: new Date().toISOString(),
  detection_type: "HUMAN_PRESENCE",
  confidence_score: 0.85,
  thumbnail_url: null,
};
```

Rule: If you find yourself copy-pasting the same mock data into multiple tests, move it to a fixture.

3.5 End-to-End (E2E) Tests (tests/e2e/tests/e2e/)

E2E tests run in a real browser using Playwright. They simulate actual user journeys from start to finish.

What we test:

- A user can complete a full task from start to finish
- Pages navigate correctly and load expected content
- Forms accept input and submit correctly
- Actions taken by one role produce the right outcome for the user

How we write them:

- Use `page.goto()` to navigate to the page
- Use `page.waitForSelector()` to wait for elements to appear
- Use `page.getByRole()` and `page.getByText()` to find and interact with elements
- Set generous timeouts (30-90 seconds) because E2E tests are slower

4. Naming Conventions

Type	Naming Pattern	Example
Unit test file	{feature}_test.py	alert_test.py
Unit test class	Test{Feature}	TestAcknowledgeAlert
Unit test method	test_{scenario}_{expected}	test_missing_alert_id_raises_400
Integration test file	{feature}_test.py	alert_test.py
Frontend unit test file	{page}.page.test.tsx	alert.page.test.tsx
E2E test file	{feature}.spec.ts	alert.spec.ts
Fixture file	{model}.ts	alert.ts

5. What Every Test Should Cover

5.1 Backend Service Tests

For every service function, write tests for:

1. **Happy path** - everything works, returns the right thing
2. **Missing required arguments** - should raise the right HTTP error (usually 400)
3. **Missing database** - should raise 500
4. **Missing or invalid auth** - should raise 401
5. **Wrong permissions** - should raise 403
6. **Resource not found** - should raise 404
7. **Conflicting state** - should raise 409
8. **Database call counts** - verify add, commit, flush, refresh, rollback were called the expected number of times

5.2 Backend Schema Tests

For every Pydantic model, write tests for:

1. **Valid construction** - all required fields present
2. **Missing each required field individually** - should raise ValidationError
3. **Invalid types** - should raise ValidationError
4. **Boundary values** - e.g. confidence score 0.0 to 1.0
5. **Optional fields** - should default to None when omitted
6. **Nested models** - should validate correctly inside wrapper responses

5.3 Integration Tests

For every API endpoint, write tests for:

1. **Successful request** - right status code and response shape
2. **Authentication required** - 401 when no auth header
3. **Related endpoints** - if endpoints are used together, test them together

5.4 Frontend Tests

For every page, write tests for:

1. **Page renders** - expected text appears
2. **User interactions** - buttons, forms, clicks work
3. **API integration** - correct functions called with correct args
4. **Error handling** - error messages appear when things go wrong
5. **Real-time updates** - WebSocket or polling updates the UI

5.5 E2E Tests

For every major user flow, write tests for:

1. **Complete journey** - from landing on the page to completing the task
2. **Expected outcomes** - the final state matches what the user should see

6. Tools and Environment

Layer	Tool	Purpose
Backend unit/integration	pytest	Run Python tests
Backend coverage	pytest-cov	Measure how much code is tested
Frontend unit	Jest	Run React component tests
Frontend coverage	Jest --coverage	Measure component test coverage
E2E	Playwright	Run browser-based tests
Mocking (Python)	unittest.mock	Fake database and external services
Mocking (JS/TS)	jest.mock()	Fake API modules and navigation

Layer	Tool	Purpose
Linting (Python)	ruff	Keep code style consistent
Linting (Frontend)	ESLint	Keep TypeScript/React code clean
Type checking	mypy	Catch type errors in Python
Security	bandit, pip-audit	Catch security issues

7. CI Pipeline

Our CI runs on every push to `main` and every pull request to `main` or `dev`. The pipeline has these jobs:

1. **Python lint** - ruff checks all Python code
2. **Frontend lint** - ESLint checks all TypeScript/React code
3. **Tests** - runs all backend unit, integration, frontend unit, and E2E tests
4. **Build** - builds the frontend to make sure it compiles
5. **Type check** - mypy checks Python types
6. **Security** - bandit and pip-audit scan for vulnerabilities

The test job uses a PostgreSQL service container and runs migrations before testing. Coverage reports are uploaded to Codecov.

8. Coverage Expectations

We are aiming for **80% code coverage** across the codebase. This is a team target, not a hard rule for every single file, but it gives us a shared goal to work toward.

Here is how that breaks down in practice:

- **Service functions:** every public function should have at least a happy path test plus error cases
- **Schemas:** every field should be tested for presence, absence, and invalid values
- **API endpoints:** every endpoint should have at least one integration test
- **Frontend pages:** every page should have at least one render test and one interaction test
- **E2E flows:** every critical user journey should have an E2E test

If you add a new feature, add tests for it in the same pull request. If your changes drop coverage below 80%, add more tests or discuss with the team whether the uncovered code is truly

untestable.

9. When to Skip Tests

Sometimes a test genuinely cannot run in CI. In that case:

- Mark it with `@pytest.mark.skip(reason="...")` and explain why
- Add a TODO comment if you plan to fix it later
- Do not skip tests just because they are failing. Fix them or remove them.

Example: `stream_test.py` skips the stream health test because OpenCV video capture cannot run in a CI container.

10. Responsibilities

Role	Responsibility
Developer writing code	Write tests for the code you add or change. Run tests locally before pushing.
Developer reviewing PR	Check that tests exist, make sense, and pass in CI.
CI pipeline	Run all tests automatically and block merges if any fail.
Team lead	Review coverage trends and decide when to invest in missing test areas.

11. Quick Reference for Writing a New Test

Python Service Test Template

```
import pytest
from unittest.mock import Mock, AsyncMock, patch
from fastapi import HTTPException
from backend.app.services.your_service import your_handler

class TestYourFeature:
    def setup_method(self):
        self.mock_db = Mock()
```



```

self.mock_db.execute = Mock()
self.mock_db.commit = Mock()
self.mock_db.rollback = Mock()
self.claims = {"sub": "test-sub", "custom:role": "RESIDENT"}

@pytest.mark.asyncio
async def test_happy_path(self):
    # Arrange
    self.mock_db.execute.return_value.scalar_one_or_none.return_value = Mock()

    # Act
    result = await your_handler(..., self.mock_db, self.claims)

    # Assert
    assert result is not None
    assert self.mock_db.commit.call_count == 1

@pytest.mark.asyncio
async def test_missing_input_raises_400(self):
    with pytest.raises(HTTPException) as exc:
        await your_handler(None, self.mock_db, self.claims)
    assert exc.value.status_code == 400

```

Python Schema Test Template

```

from pydantic import ValidationError
import pytest
from app.schemas.your_schema import YourReq

class TestYourReq:
    def test_valid_request(self):
        req = YourReq(field="value")
        assert req.field == "value"

    def test_missing_field_raises_validation_error(self):
        with pytest.raises(ValidationError):
            YourReq()

```

Frontend Unit Test Template

```

import React from "react";
import "@testing-library/jest-dom";
import { render, screen, fireEvent, waitFor } from "@testing-library/react";
import YourPage from "@app/your-page/page";

```

```
import * as api from "@lib/api/yourApi";

jest.mock("@lib/api/yourApi", () => ({
  yourFunction: jest.fn(),
}));

const mocked = api as jest.Mocked<typeof api>;

describe("YourPage", () => {
  beforeEach(() => {
    jest.clearAllMocks();
  });

  it("renders and interacts correctly", async () => {
    mocked.yourFunction.mockResolvedValue({} as any);
    render(<YourPage />);
    expect(await screen.findByText(/expected/i)).toBeInTheDocument();
  });
});
```

E2E Test Template

```
import { test, expect } from "@playwright/test";

test("your user flow", async ({ page }) => {
  await page.goto("/your-page");
  await page.waitForSelector("#your-element", { timeout: 10000 });
  await page.locator("#your-element").fill("value");
  await page.getByRole("button", { name: /submit/i }).click();
  await expect(page.getByText(/success/i)).toBeVisible();
});
```

12. Final Notes

- If a test is hard to write, that usually means the code is too complicated. Simplify the code first.
 - Do not write tests just to hit a coverage number. Write tests that would have caught a real bug.
 - When in doubt, ask. It is better to spend five minutes discussing a test than to ship broken code.
 - This document will evolve. If something here does not match reality, update it.
-

End of document